Technical Blueprint

Metro Ticket Generating System in ServiceNow | Technical documentation | 11 October 2026

This blueprint records the ServiceNow components, identifiers, data path, and code used in the capstone. The active demonstration path is catalog variables to client scripts to a Service Portal QR widget.

1 Component Architecture

Layer	Configured component	Role
Presentation	Service Portal catalog item Book a Ticket	Collect route, journey, passenger, and payment mode choices.
Reference data	Metro Station Details table	Supply station names for the two reference variables.
Client logic	Fare auto Calculation onChange script	Calculate a fare from the 27 station matrix.
Conditional UI	Fields Visibility catalog UI policy	Display Enter Payment Mode when Others is selected.
QR display	QRGeneration onSubmit and Metro QR Widget	Construct and display the exercise QR image.

Configured path: Service Portal form -> catalog variables -> fare script/UI policy -> QRGeneration -> spModal -> metro_qr_widget. The catalog record currently shows the DEFAULT execution plan in Process Engine.

2 Data Model and Mapping

The custom table is Metro Station Details [u_metro_station_details]. Its user-created station field is Station Name [u_station_name]. The Starting From and Going To catalog variables reference this table; the fare script resolves each selected record to u_station_name before matching it to the route matrix.

Source	Target or use
u_metro_station_details.u_station_name	Display text for source and destination references
starting_from / going_to	Reference record IDs read by g_form.getValue and resolved with g_form.getReference
type_of_jouney	Choice value 1 for single, 2 for return
number_of_passengers	Multiplier in fare calculation
amount_for_single_journey	One way amount shown on the form
amount_including_return	Return amount shown on the form
mode_of_payment	Condition input for Fields Visibility
enter_payment_mode	Additional field shown for Others

3 Fare Calculation

Fare auto Calculation is an active onChange catalog client script bound to number_of_passengers. It reads the two reference IDs, journey type, and passenger count. It normalizes station names, finds their positions in a 27 by 27 printed-fare matrix, maps that value to a stored fare value, then multiplies by passenger count. The return amount doubles the single journey calculation. The complete instance script is saved in 5. Project Development Phase/fare-auto-calculation.js.

```
var printedFare = Number(printedFareRows[from].split(' ')[to]);
var oneWayFare = currentFareByPrintedFare[printedFare];
if (journeyType === '1') {
    g_form.setValue('amount_for_single_journey', String(oneWayFare * passengers));
} else {
    g_form.setValue('amount_including_return', String(oneWayFare * passengers * 2));
}
```

4 QR Widget and Script

The Metro QR Widget has ID metro_qr_widget. Its server script reads input.qr_url, and its HTML template binds that URL to an image. QRGeneration constructs a demonstration URL using example.com, sends it to qrserver.com for image generation, and presents the widget in spModal. The onSubmit script returns false, so this configured path presents a QR preview rather than a completed request. The exact scripts and template are included as source snapshots.

```
var dummyURL = 'https://example.com/ticket?id=' + sysId;
var qrImageUrl = 'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=' + encodeURIComponent(dummyURL);
spModal.open({
    title: 'Your Metro QR Ticket',
    widget: 'metro_qr_widget',
    widgetInput: { qr_url: qrImageUrl }
});
```

5 Access and Configuration Inventory

- Catalog item: Book a Ticket; Global application; Service Catalog; Services category.
- Variables: eight ordered fields and one catalog UI policy.
- Catalog client scripts: Fare auto Calculation and QRGeneration, both active.
- Station table: 27 records; default table ACLs described in the Skill Wallet access-control task.
- Widget: Metro QR Widget, ID metro_qr_widget, Global application.